StockPulse: Microservices Inventory Management System

Siya Patel

LogicVeda Web Development Domain

Executive Summary

StockPulse is a microservices-based inventory and order management platform designed to demonstrate modern backend architecture principles used in large-scale retail and e-commerce systems. The platform separates authentication, inventory management, order processing, and API routing into independent services while providing centralized monitoring through a React-based administrative dashboard.

The project focuses on scalability, resilience, observability, and maintainability. It incorporates JWT-based authentication, event-driven order workflows, service health monitoring, Dockerized deployment, load testing, and failure recovery mechanisms to simulate real-world production environments.

Project Objective

The objective of this project was to design and implement a distributed inventory management system capable of handling inventory tracking, order processing, authentication, and service monitoring through independently deployable services.

The system was built to reflect industry practices commonly used in modern SaaS platforms and enterprise retail applications where reliability, scalability, and fault tolerance are critical.

Business Value

StockPulse addresses several common challenges found in traditional monolithic retail systems:

- Independent service deployment and maintenance
- Improved scalability through service separation
- Better fault isolation during failures
- Centralized monitoring and visibility
- Secure user authentication and authorization
- Real-time inventory and order tracking

By separating business functions into dedicated services, the platform becomes easier to extend, test, and operate.

System Architecture

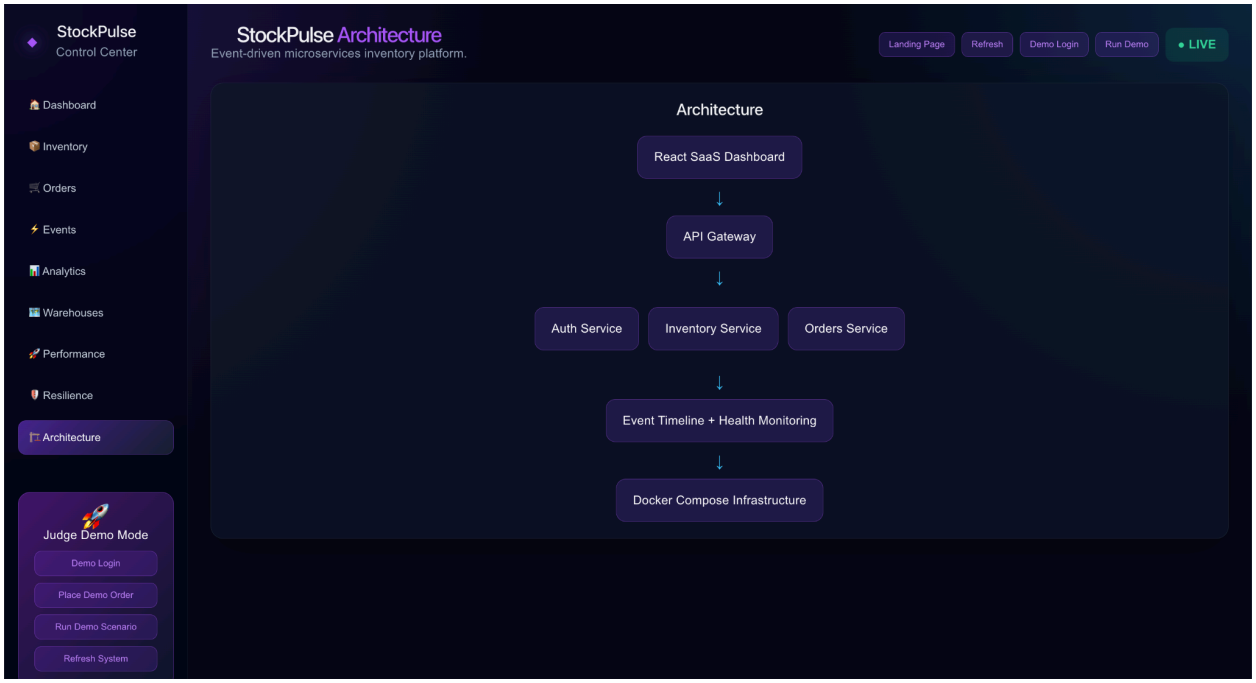

Figure 1. High-level microservices architecture of StockPulse.

The platform consists of five primary components:

React Dashboard

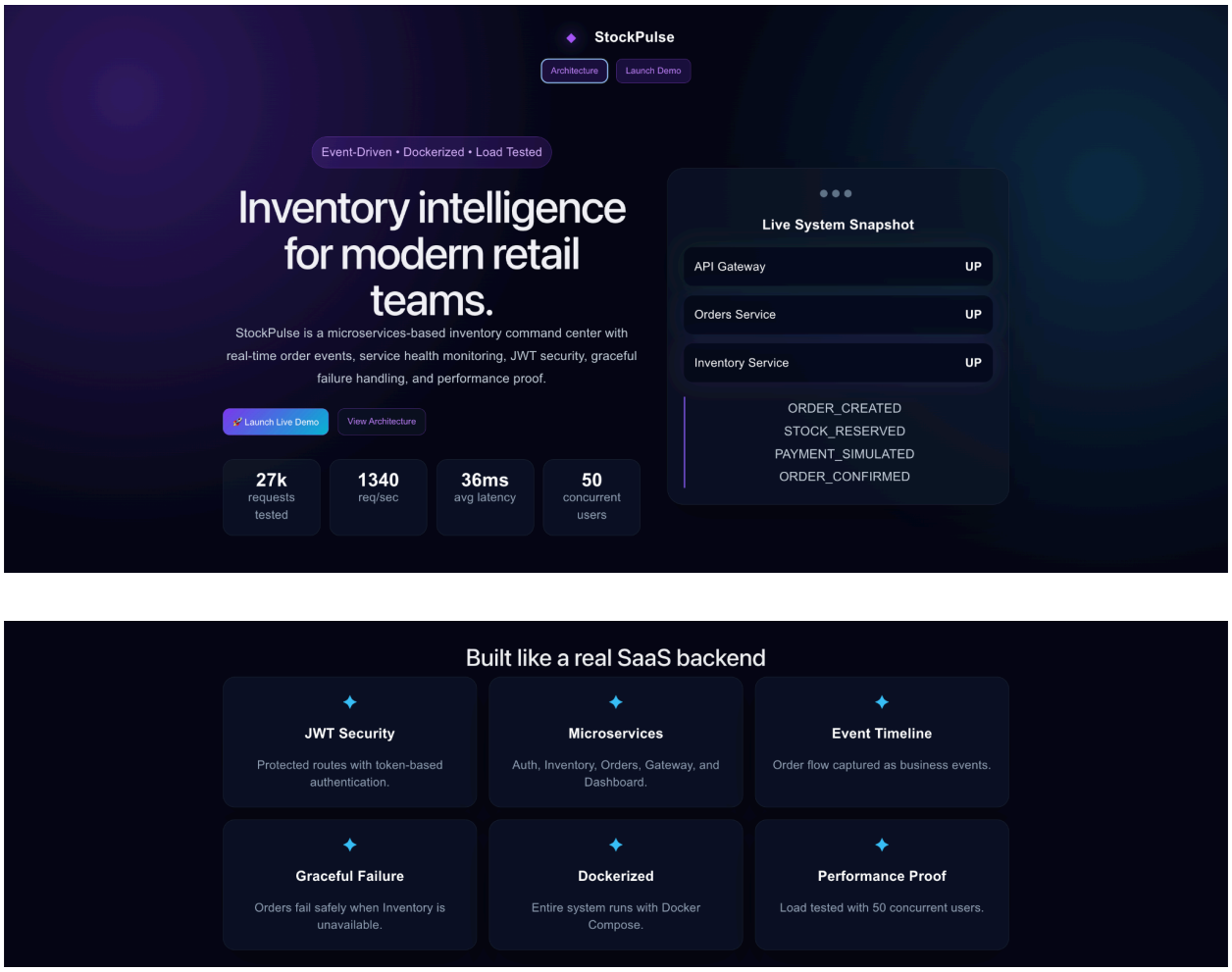

Figure 2. StockPulse landing page used as the evaluator entry point.

Provides administrators with visibility into inventory levels, order activity, service health, analytics, and operational metrics.

API Gateway

Acts as a centralized entry point for all client requests and routes traffic to the appropriate microservice.

Authentication Service

Handles user registration, login, JWT generation, and token validation for protected routes.

Inventory Service

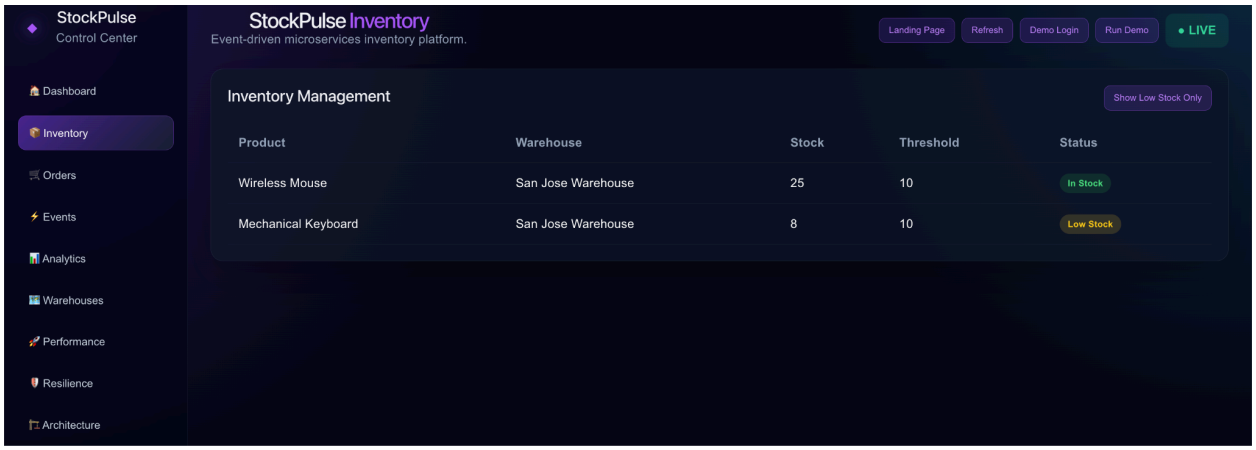

Figure 3. Inventory management interface with stock tracking and low-stock monitoring.

Manages product data, stock levels, inventory updates, and low-stock monitoring.

Orders Service

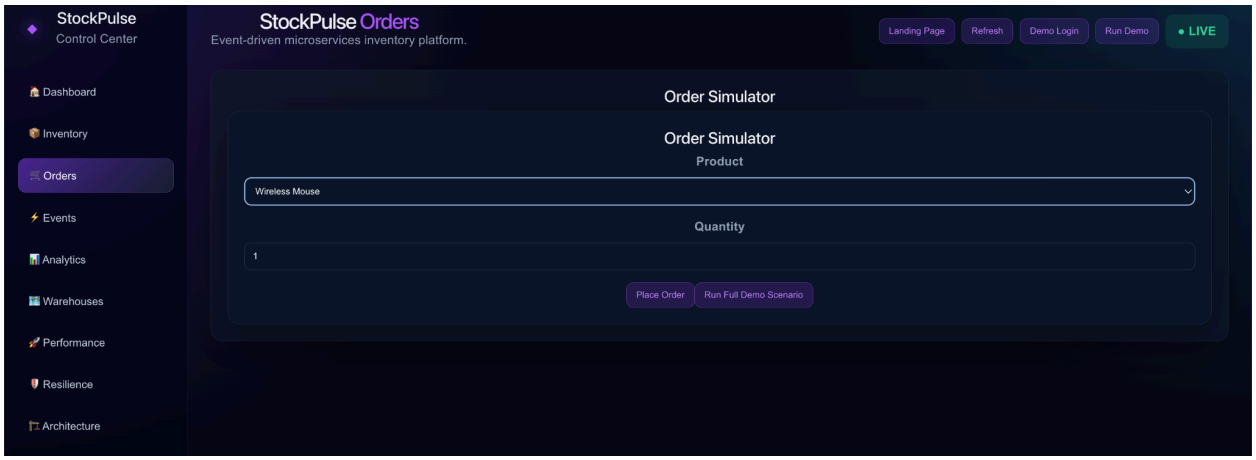

Figure 4. Interactive order simulator used for testing order workflows.

Processes customer orders, coordinates inventory deductions, and records order lifecycle events.

The overall architecture follows the structure below:

React Dashboard → API Gateway → Microservices

The API Gateway communicates with the Authentication, Inventory, and Orders services while exposing a unified interface to the frontend.

Core Features

Authentication & Authorization

- User registration and login
- JWT token generation
- Protected API endpoints
- Token validation across services

Inventory Management

- Product inventory tracking
- Stock updates
- Low-stock alerts
- Warehouse assignment

Order Processing

- Order creation workflow
- Inventory reservation
- Payment simulation
- Order confirmation

Event Tracking

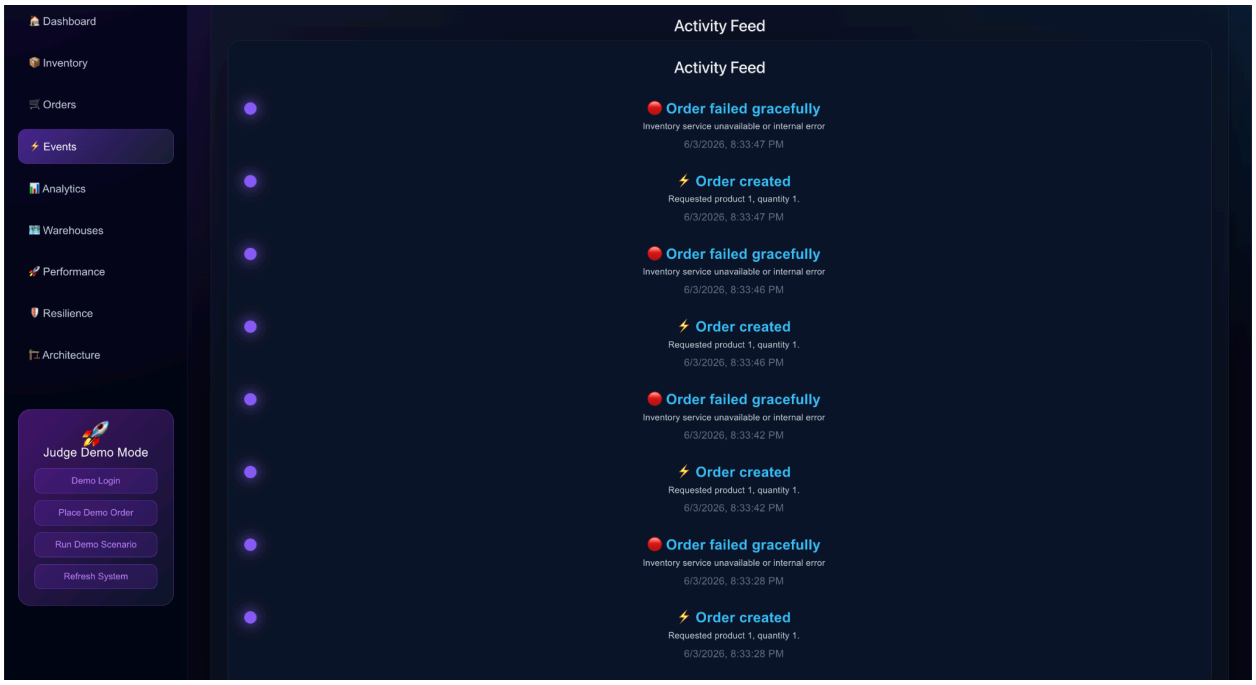

Figure 5. Event timeline capturing the complete order lifecycle.

Every order generates a sequence of events:

- ORDER_CREATED
- STOCK_RESERVED

- PAYMENT_SIMULATED
- ORDER_CONFIRMED

Failure scenarios generate:

- ORDER_FAILED

These events are displayed through the dashboard activity feed to provide transparency into system behavior.

Service Health Monitoring

Each microservice exposes health endpoints that are aggregated by the API Gateway and displayed through the dashboard.

Judge Demo Mode

A one-click demonstration mode automatically performs login, order placement, inventory updates, and event generation to simplify project evaluation.

Technical Stack

Category	Technology
Frontend	React, Vite
Backend	Node.js, Express
Authentication	JWT, bcrypt
API Communication	REST APIs, Axios
Containerization	Docker, Docker Compose

Testing	Postman, Autocannon
Architecture	Microservices

Performance Testing

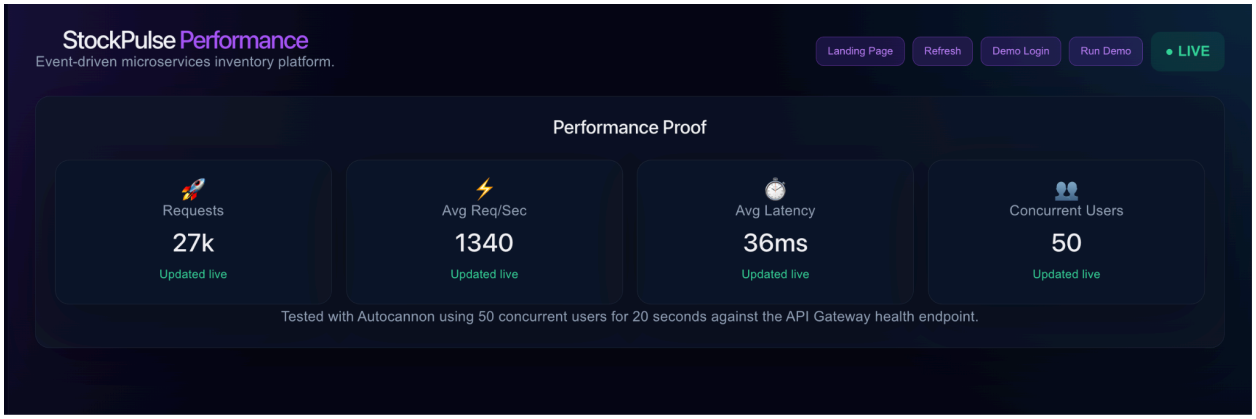

Figure 6. Autocannon load test results validating system performance and scalability.

System performance was validated using Autocannon against the API Gateway.

Test Configuration:

- 50 concurrent users
- 20-second test duration

Results:

- Over 27,000 requests processed
- Approximately 1,340 requests per second
- Average latency of 36 milliseconds
- No service failures during testing

These results demonstrate the platform's ability to handle concurrent traffic while maintaining low response times.

Resilience Testing

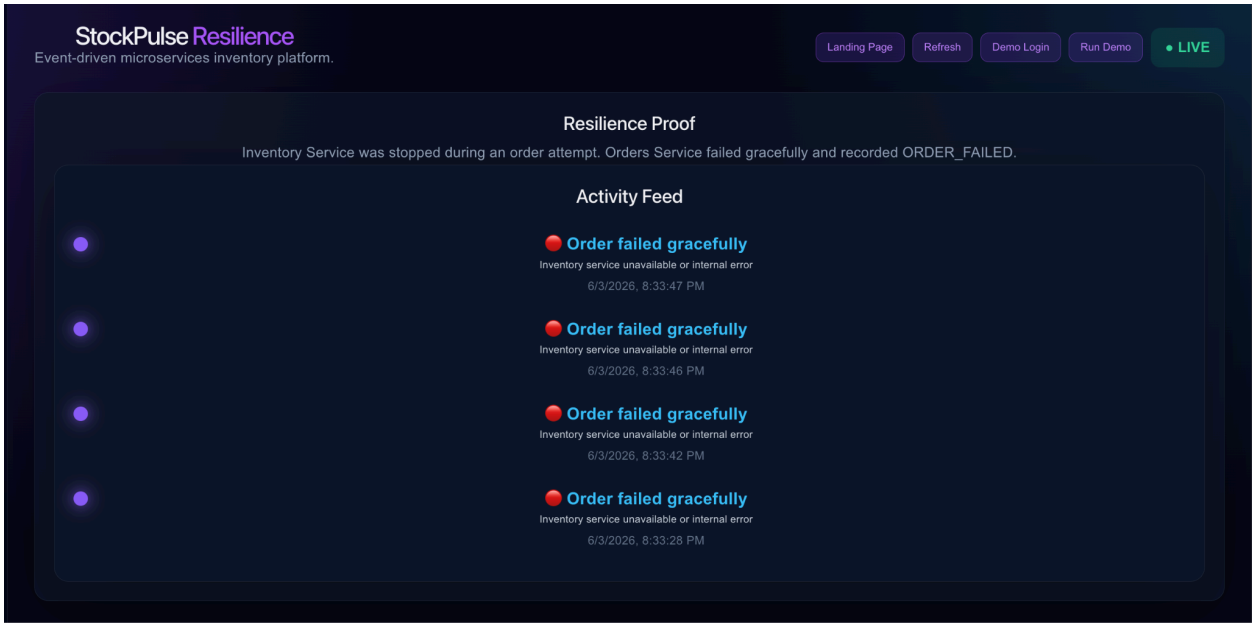

Figure 7. Graceful failure handling when Inventory Service becomes unavailable.

A controlled failure test was conducted by intentionally stopping the Inventory Service during order execution.

Expected Behavior:

- Orders should fail safely
- System should remain operational
- Failure should be recorded

Observed Behavior:

- Order requests returned graceful failure responses
- ORDER_FAILED events were recorded
- Other services continued operating normally
- Successful processing resumed immediately after service restoration

This validates the platform's ability to tolerate partial system failures without causing complete service disruption.

Dashboard Overview

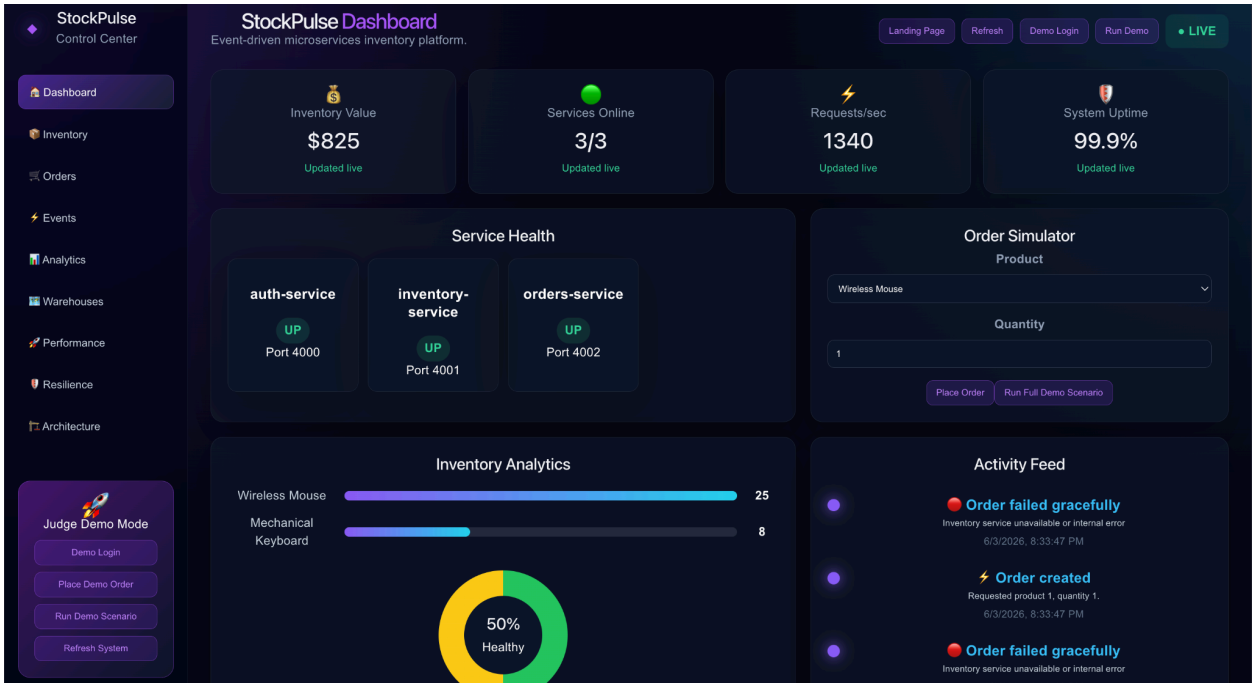

Figure 8. Executive operations dashboard displaying real-time metrics.

The administrative dashboard was designed to resemble a modern SaaS operations platform and includes:

- Landing page
- Executive metrics
- Service health monitoring
- Inventory analytics
- Activity feed
- Order simulator
- Warehouse network visualization
- Performance metrics
- Resilience testing results
- Architecture overview

The dashboard provides both technical visibility and business-focused insights.

Key Challenges

The primary challenge was coordinating communication between independent services while maintaining reliability during failures.

This was addressed through:

- API Gateway routing

- JWT-based service security
- Health monitoring endpoints
- Event-driven workflows
- Graceful error handling

Another challenge was proving system reliability rather than simply implementing features. Load testing, resilience testing, Docker deployment, and operational monitoring were incorporated to provide measurable evidence of system behavior.

Deployment & Operations

The entire platform was containerized using Docker and Docker Compose. All services, including the React dashboard, API Gateway, Authentication Service, Inventory Service, and Orders Service, can be launched together through a single command.

This approach simplifies deployment, ensures consistent development environments, and demonstrates containerized infrastructure practices commonly used in production systems.

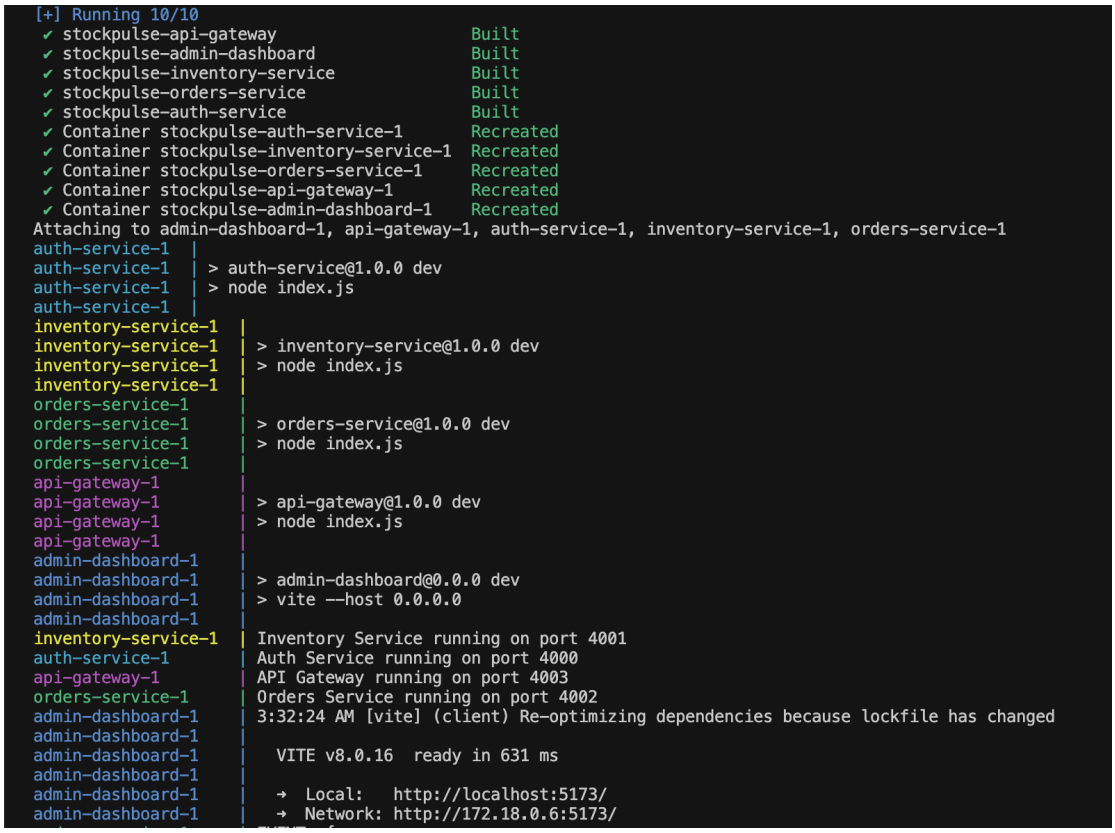

Figure 9. Docker Compose deployment running all microservices and frontend components.

Lessons Learned

This project provided practical experience with distributed systems architecture, API design, authentication workflows, service communication, operational monitoring, and containerized deployments.

The most significant takeaway was understanding that production-quality systems require more than functional features. Reliability, observability, testing, and documentation are equally important components of professional software engineering.

Future Enhancements

Potential future improvements include:

- PostgreSQL persistence layer
- Kafka or RabbitMQ event streaming
- Kubernetes orchestration
- Elasticsearch integration
- Role-based access control
- CI/CD automation
- Cloud deployment infrastructure

Conclusion

StockPulse successfully demonstrates the design and implementation of a modern microservices-based inventory management platform. Through secure authentication, distributed services, event-driven workflows, performance validation, resilience testing, and a polished administrative interface, the project reflects many of the architectural patterns used in contemporary enterprise software systems.

The result is a scalable, maintainable, and production-inspired platform that showcases both technical depth and practical software engineering principles.

GitHub Repository:

https://github.com/siyaapatel06/stockpulse-microservices-inventory